CSE 554 Machine Learning Systems
Lecture 11 - Mixture of Experts

Michael Gu
Baris Kasikci

Contents

- MoE architecture
  - Overview
  - MoE layer
  - Routing
- Training algorithms
- System optimizations
  - Training
  - Deployment

Mixture of experts

Mistral AI
@MistralAI

magnet:?
xt=urn:btih:9238b09245d0d8cd915be09927769d5f7584c1c9&dn=mixtral-8x22b&tr=udp%3A%2F%2Fopen.demonii.com%3A1337%2Fannounce&tr=http%3A%2F%2Ftracker.opentrackr.org%3A1337%2Fannounce

deepseek

DeepSeekMoE: Towards Ultimate Expert Specialization in Mixture-of-Experts Language Models

Grok

Llama 4:
Leading Multimodal Intelligence

Newest model suite offering unrivale speed and efficiency

Llama 4 Behemoth
288B active parameter, 16 experts
2T total parameters
The most intelligent teacher model for distillation
Preview

Llama 4 Maverick
17B active parameters, 128 experts
400B total parameters
Native multimodal with 1M context length

Llama 4 Scout

deepseek

DeepSeek-V3 Technical Report

Gemini 3

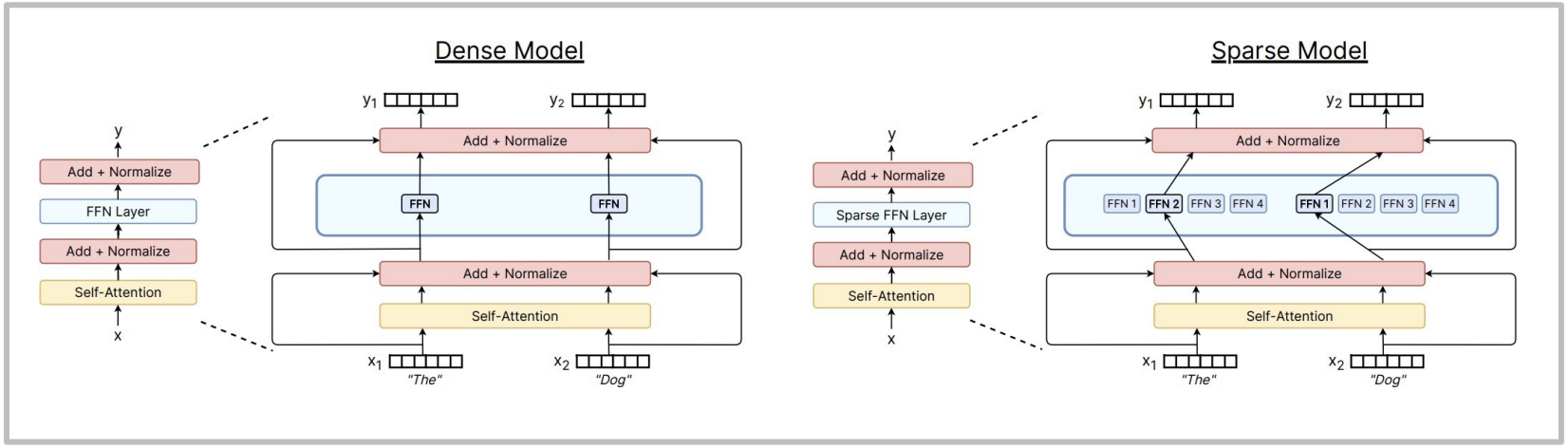
OLMoE: Open Mixture-of-Experts Language Models

Niklas Muennighoff[ca] Luca Soldaini[a] Dirk Groeneveld[a] Kyle Lo[a] Jacob Morrison[a]
Sewon Min[a] Weijia Shi[w] Pete Walsh[a] Oyvind Tafjord[a] Nathan Lambert[a]
Yuling Gu[a] Shane Arora[a] Akshita Bhagia[a] Dustin Schwenk[a] David Wadden[a]
Alexander Wettig[ap] Binyuan Hui Tim Dettmers[a] Douwe Kiela[c] Ali Farhadi[aw]
Noah A. Smith[aw] Pang Wei Koh[aw] Amanpreet Singh[c] Hannaneh Hajishirzi[aw]
[a]Allen Institute for AI [c]Contextual AI [w]University of Washington [p]Princeton University
n.muennighoff@gmail.com hannah@allenai.org

GPT-5

Gemma ~~3~~ 4?

MoE at a glance



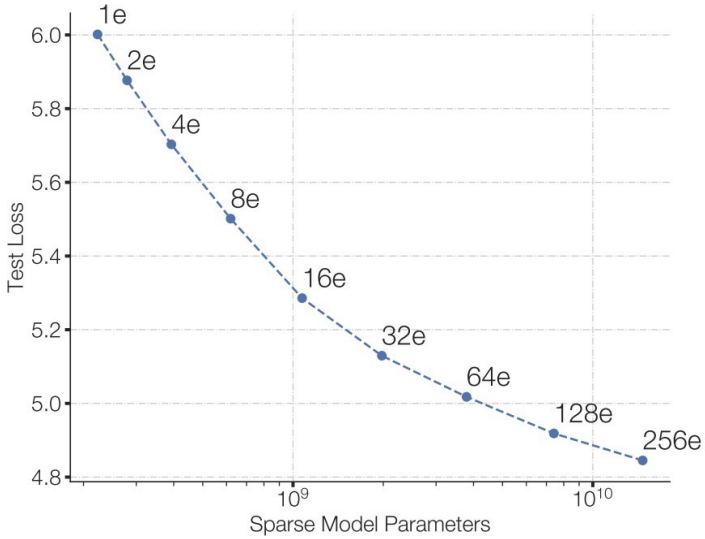
[Switch Transformers]

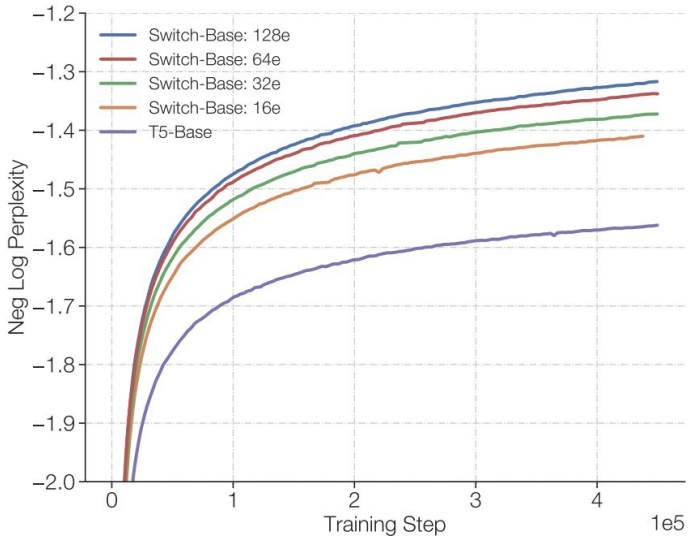
Replace big feedforward with (**many**) big feedforward networks and a **selector** layer

You can increase the # experts without affecting FLOPs

Why are MoEs getting popular?

Same FLOP, more param does better

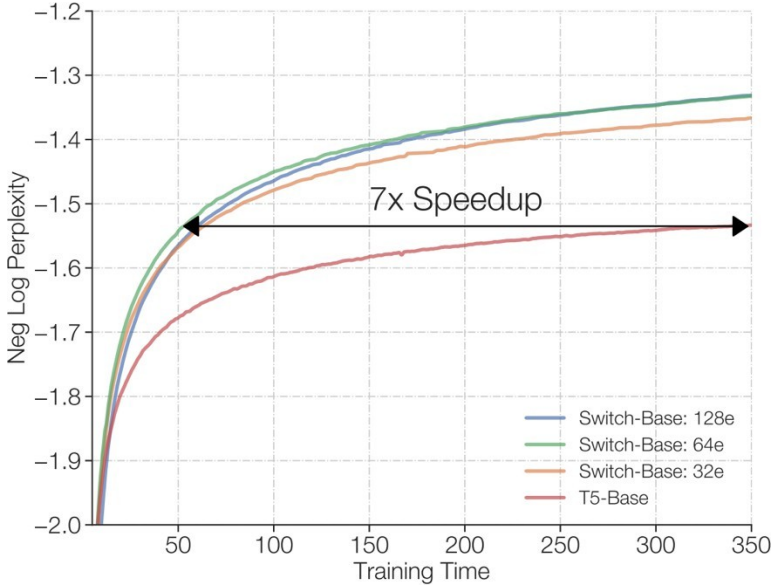




[Switch Transformers]

Why are MoEs getting popular?

Faster to train MoEs



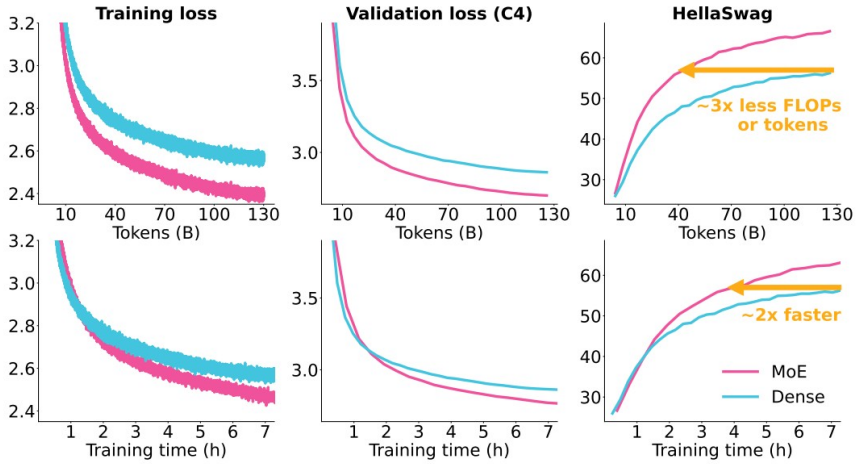
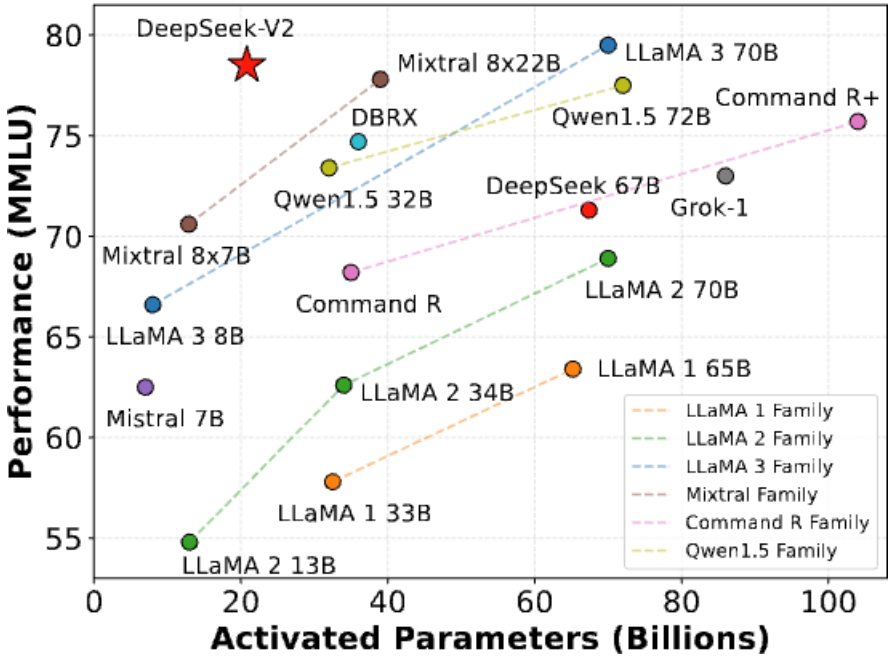


Figure 4: **MoE vs. Dense.** We train a 1.3B parameter dense model and a 1.3B active, 6.9B total parameter MoE model, each on 128 H100 GPUs. Apart from MoE-related changes, we train both with the same configuration for 130B tokens. The MoE contains 64 experts out of which 8 are

[Switch Transformers]

[OlMoE]

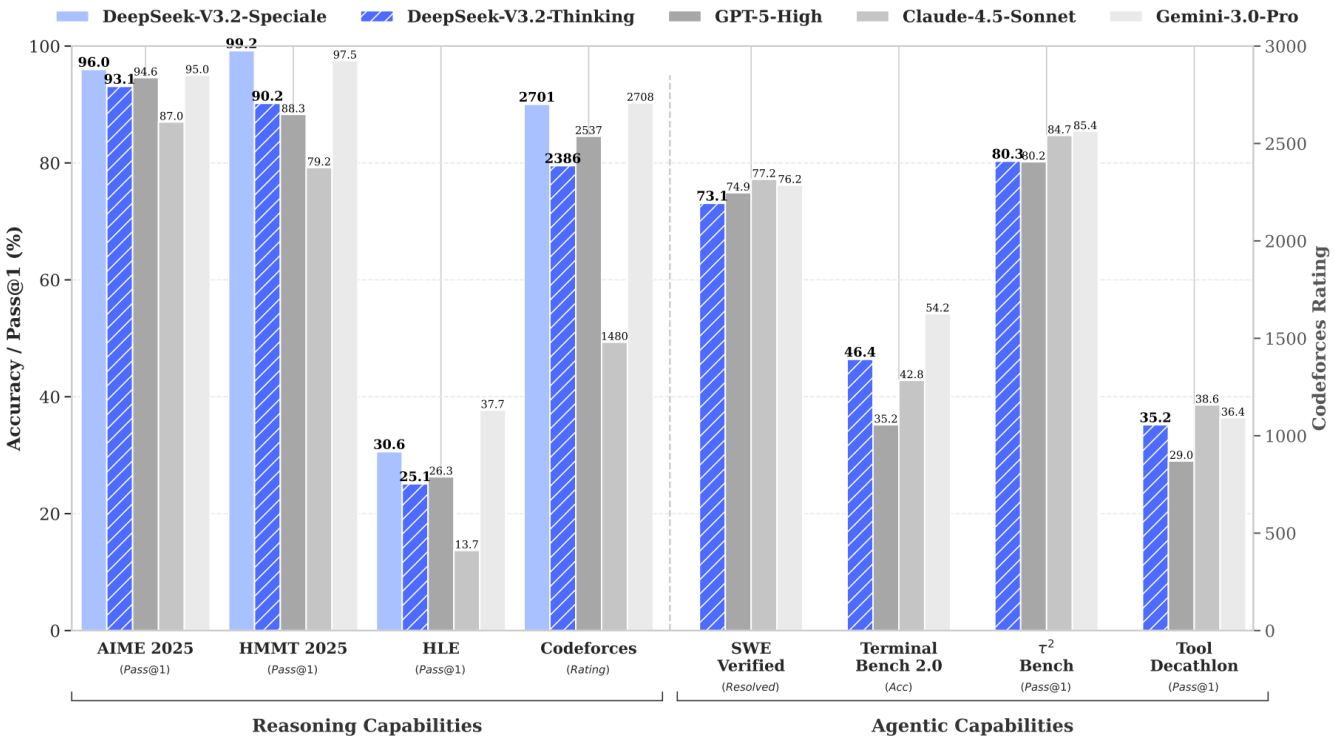
Why are MoEs getting popular?



[DeepSeek V2]

Highly competitive vs dense equivalents

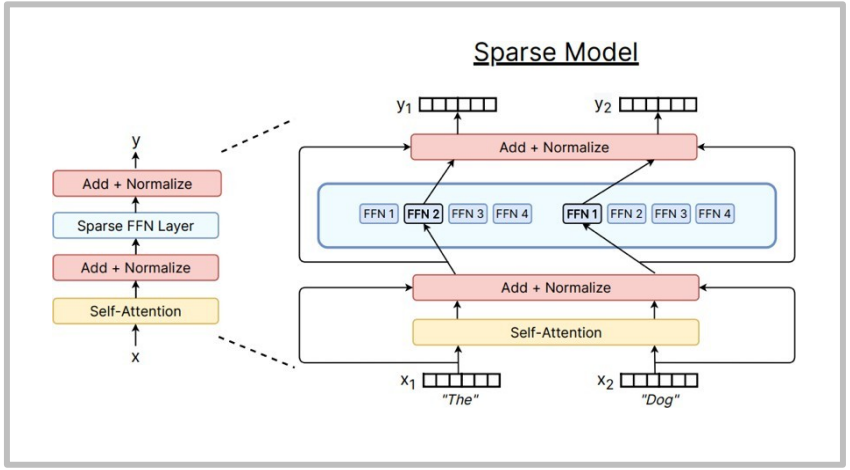
Recent MoE results – DeepSeek v3.2



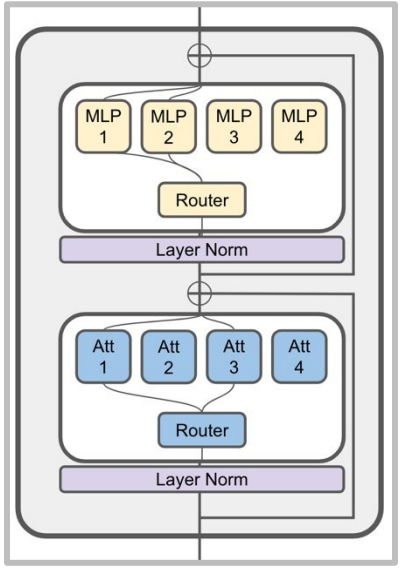
[DeepSeek V3.2]

What MoEs generally look like

Typical: replace MLP with MoE layer



Less common: MoE for attention heads



[JetMoE]

MoE – what varies?

- Routing function
- Expert sizes
- Training objectives

Routing function - overview

How to create this token-expert mapping?

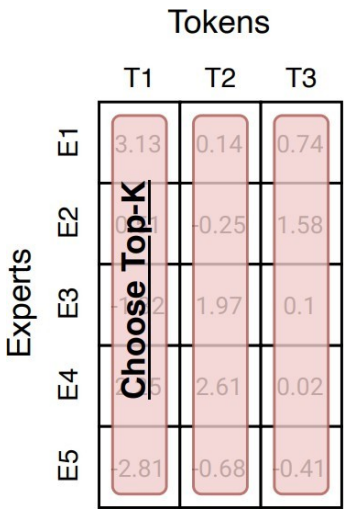
First calculate a map of token-expert scores

Token chooses expert

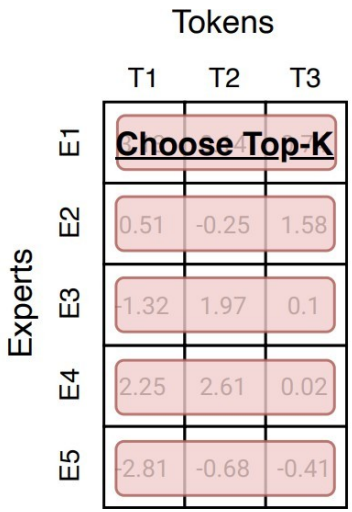
- Each token independently selects one or more experts to be processed
- **top-k**: how many experts to choose per token

Expert chooses token

- Each expert decides which tokens it wants to process
- **capacity factor**: how many tokens to choose per expert



Token chooses expert



Expert chooses token

[Switch Transformers]

Routing type

Almost all the MoEs do a standard ‘token choice topk’ routing. Some recent ablations

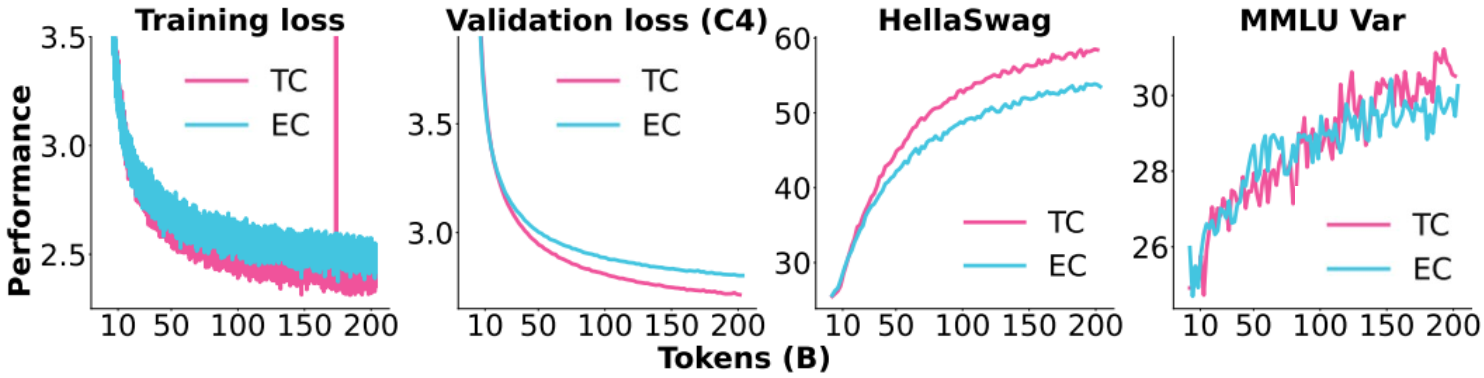
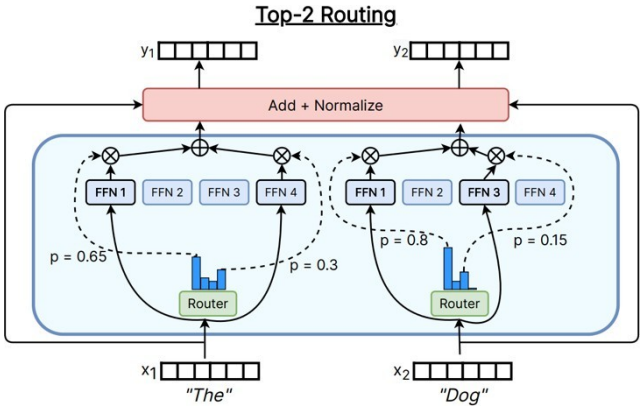


Figure 7: **Expert choice (EC) vs. token choice (TC).** Both models have an 8-expert MoE in every 2nd layer. For TC, 2 experts are activated per token, while for EC the capacity factor is 2. Thus, both models use the same number of active parameters. More results, logs, and configurations: https://wandb.ai/ai2-llm/olmoe/reports/Plot-EC-vs-TC--Vmlldzo4MzkzMDM3

[OlMoE]

Common routing variants in detail

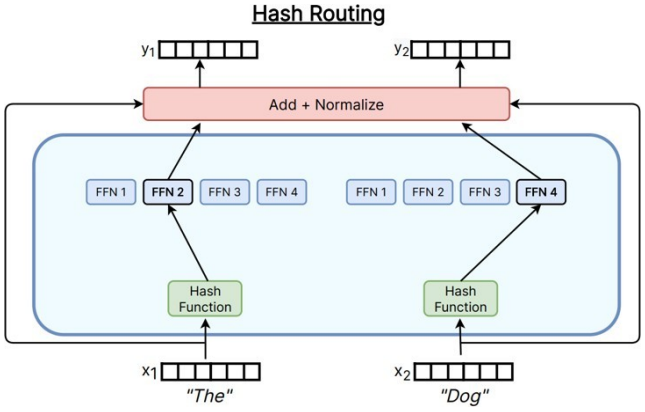
Top-k



Used in *most* MoEs

Switch Transformer (k=1)
Gshard (k=2), Grok (2),
Mixtral (2),
Qwen (4), DBRX (4),
DeepSeek (7)

Hashing



Common baseline

[Switch Transformers]

Top-K routing in detail.

Most papers do the old and classic top-k routing. How does this work?

Gating

$$\mathbf{h}_t^l = \sum_{i=1}^{N} \left(g_{i,t} \, \mathrm{FFN}_i \left(\mathbf{u}_t^l \right) \right) + \mathbf{u}_t^l,$$

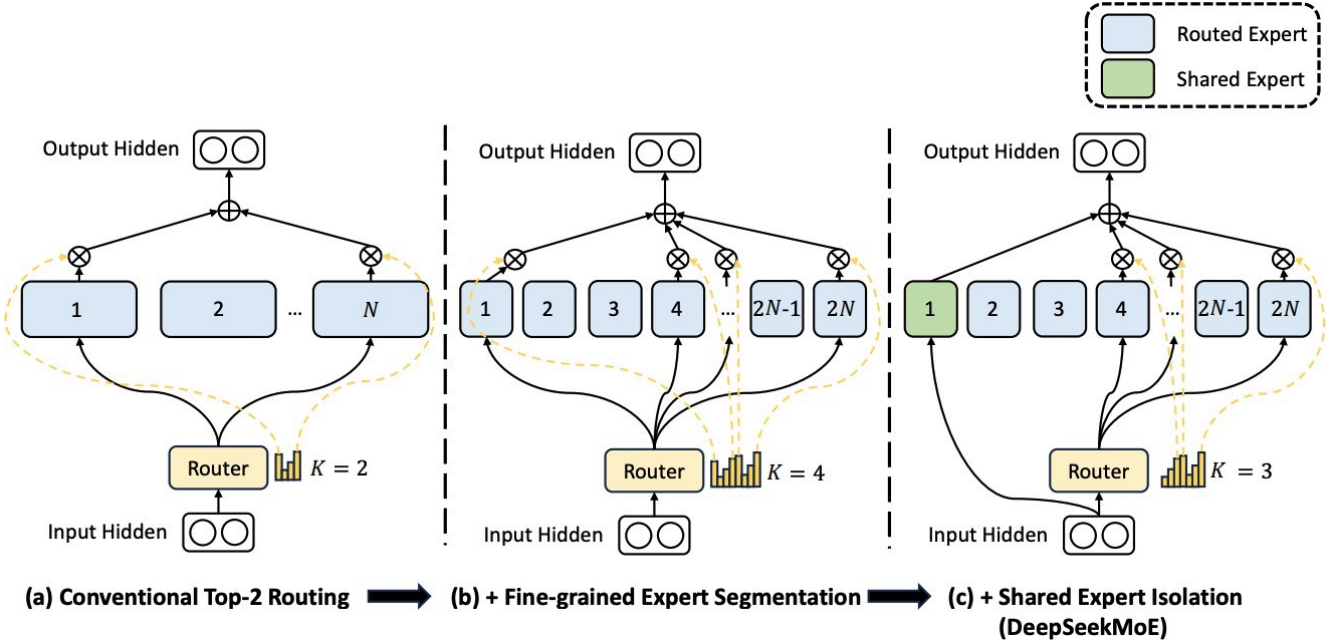
$$g_{i,t} = \begin{cases} s_{i,t}, & s_{i,t} \in \mathrm{Topk}(\{s_{j,t} | 1 \leqslant j \leqslant N\}, K), \\ 0, & \text{otherwise}, \end{cases}$$

$$s_{i,t} = \mathrm{Softmax}_i \left(\mathbf{u}_t^l \mathbf{W}_g \right),$$

This is the DeepSeek (V1-2) router
(Grok, Qwen do this too)

Mixtral, DBRX, DeepSeek v3 softmaxes after the TopK

Recent variations from DeepSeek/Qwen



Smaller, larger number of experts + a few shared experts that are always on.

(Used in DeepSeek / Qwen, originally from DeepSpeed MoE)

Various ablations from the DeepSeek paper

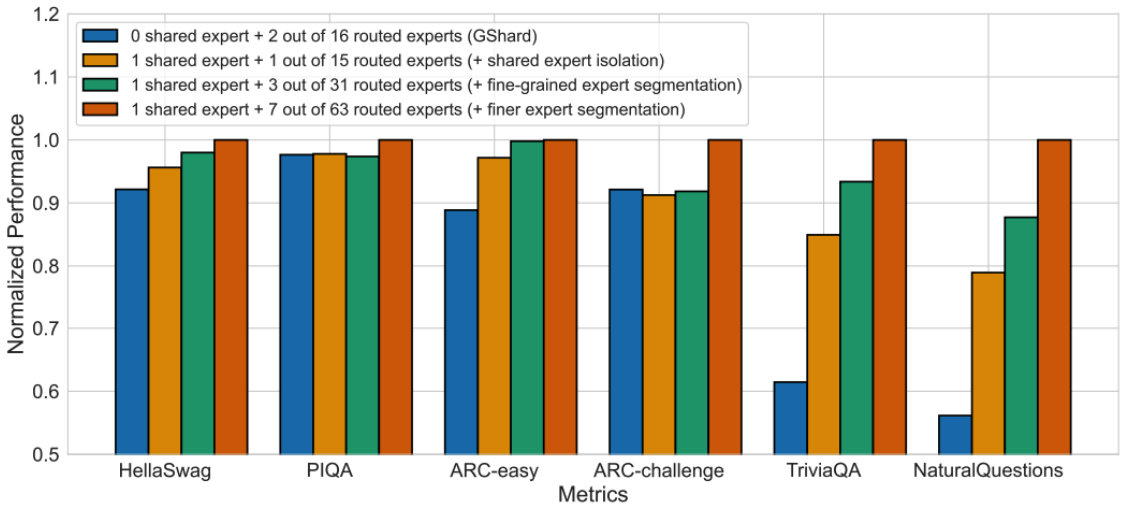


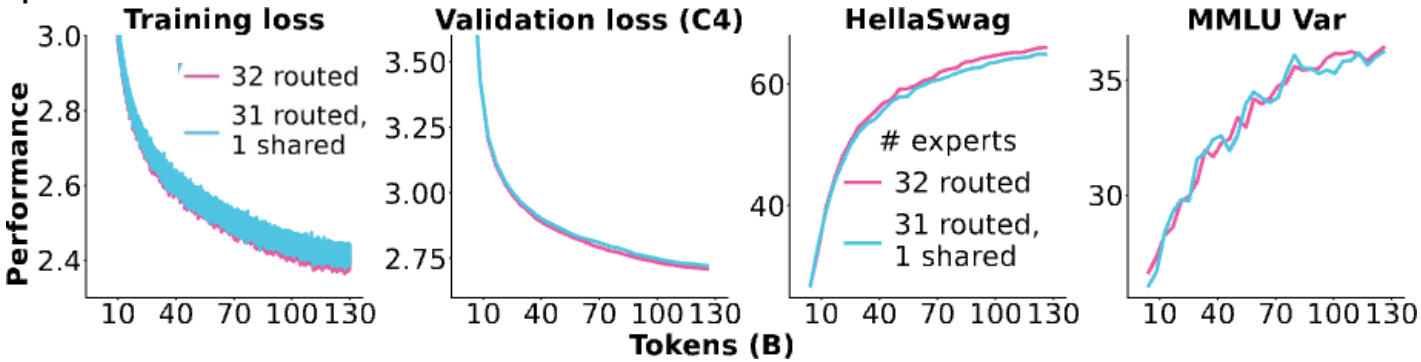
Figure 3 | Ablation studies for DeepSeekMoE. The performance is normalized by the best performance for clarity in presentation. All compared models have the same number of parameters and activated parameters. We can find that fine-grained expert segmentation and shared expert isolation both contribute to stronger overall performance.

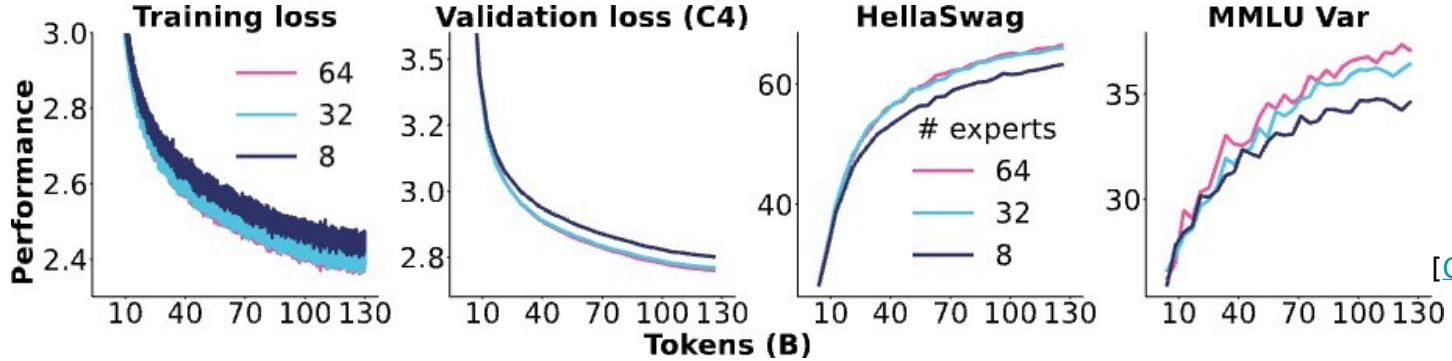
More experts, shared experts all seem to generally help

[DeepSeek-MoE]

Ablations from OlMoE

Gains from fine-grained experts, none from shared experts.





[OlMoE]

Expert routing setups for recent MoEs

Model	Routed	Active	Shared	Fine-grained ratio
GShard	2048	2	0	
Switch Transformer	64	1	0	
Mixtral	8	2	0	
DBRX	16	4	0	
Grok	8	2	0	
DeepSeek v1	64	6	2	1/4
Qwen 1.5	60	4	4	1/8
DeepSeek v3	256	8	1	1/14
OlMoE	64	8	0	1/8
MiniMax	32	2	0	~1/4
Llama 4 (maverick)	128	1	1	1/2
Qwen3	128	8	0	1/4

How do we train MoEs?

Major challenge: we need sparsity for training-time efficiency…

But sparse gating decisions are not differentiable!

After the top-k mask, backpropagation can only update gradients for the selected experts…

Solutions?

1. Reinforcement learning to optimize gating policies
2. Stochastic perturbations
3. **Heuristic ‘balancing’ losses.**

Load balancing ablation

Without load balancing, router will only route to 2 experts :(

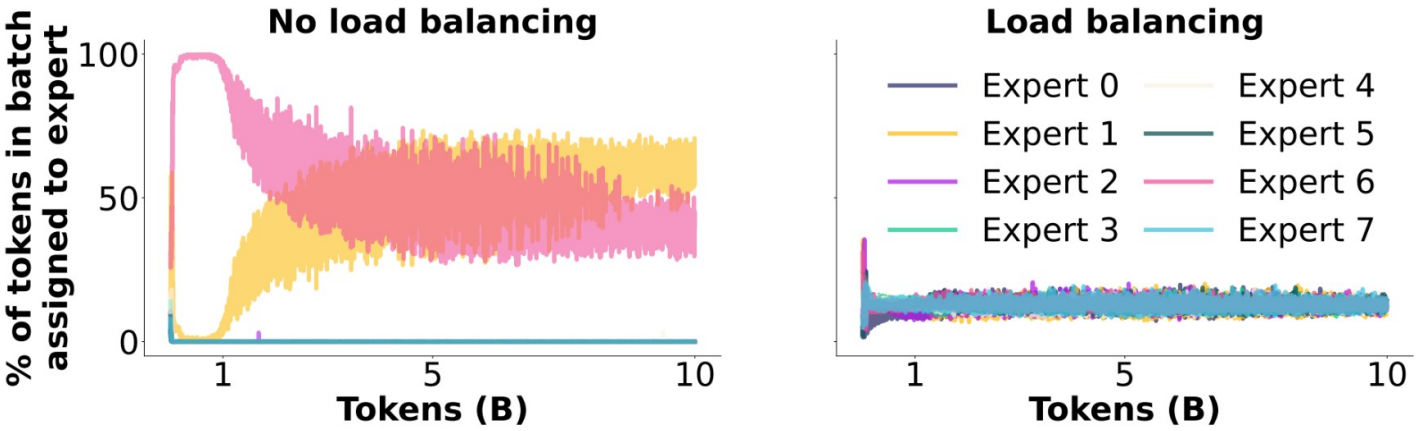


Figure 10: **Expert assignment during training when using or not using a load balancing loss for the first MoE layer.** More results, logs, and configurations: https://wandb.ai/ai2-llm/olmoe/reports/Plot-LBL-vs-No-LBL--Vmlldzo4OTkyNDg4

[OlMoE]

Example from deepseek (v1-2)

Per-expert balancing – same as the switch transformer

$$\mathcal{L}_{\text{ExpBal}} = \alpha_1 \sum_{i=1}^{N'} f_i P_i, \tag{12}$$

$$f_i = \frac{N'}{K'T} \sum_{t=1}^{T} \mathbb{1}(\text{Token } t \text{ selects Expert } i), \tag{13}$$

$$P_i = \frac{1}{T} \sum_{t=1}^{T} s_{i,t}, \tag{14}$$

[DeepSeek-MoE]

Per-device balancing – the objective above, but aggregated by device.

$$\mathcal{L}_{\text{DevBal}} = \alpha_2 \sum_{i=1}^{D} f'_i P'_i, \tag{15}$$

$$f'_i = \frac{1}{|\mathcal{E}_i|} \sum_{j \in \mathcal{E}_i} f_j, \tag{16}$$

$$P'_i = \sum_{j \in \mathcal{E}_i} P_j, \tag{17}$$

[DeepSeek-v2]

[DeepSeek-MoE]

DeepSeek v3 variation – per-expert biases

Set up a per-expert bias (making it more likely to get tokens) and use online learning

$$g'_{i,t} = \begin{cases} s_{i,t}, & s_{i,t} + b_i \in \mathrm{Topk}(\{s_{j,t} + b_j | 1 \leqslant j \leqslant N_r\}, K_r), \\ 0, & \text{otherwise.} \end{cases}$$

[DeepSeek-v3]

Qwen3-MoE Global Batch Load Balancing

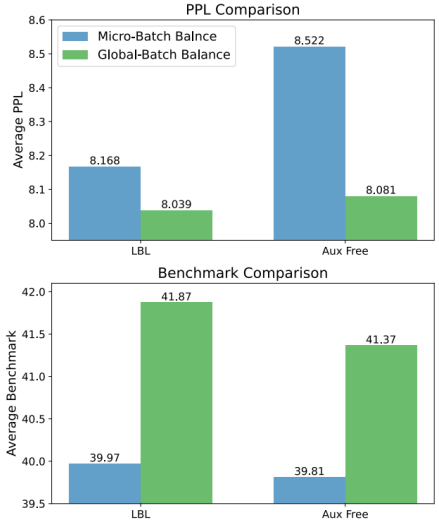
Doing load balancing in a micro-batch could limit expert specialization
Instead, do global-batch load balancing across all parallel device groups

$$\mathrm{LBL}_{\mathrm{micro}} = \frac{1}{N_P}\sum_{j=1}^{N_p}(N_E\sum_{i=1}^{N_E} f_i^j \cdot P_i^j), \quad (3)$$

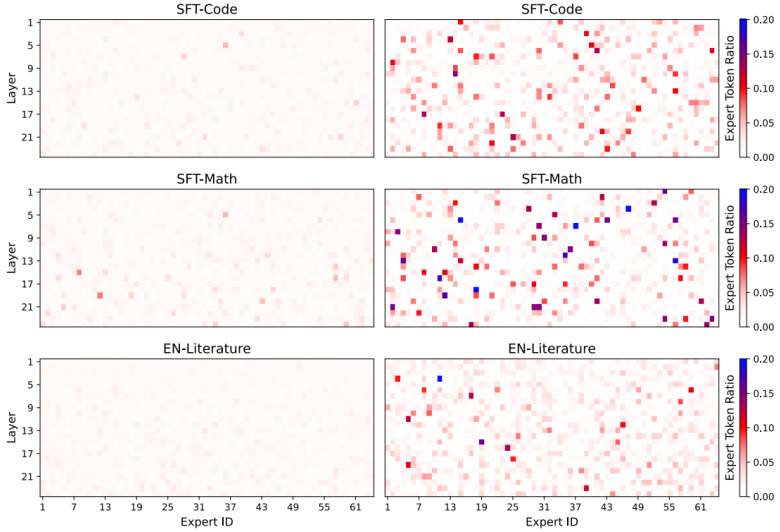
$$\mathrm{LBL}_{\mathrm{global}} = N_E\sum_{i=1}^{N_E} \bar{f}_i \cdot \bar{P}_i \quad (4)$$

$$= N_E\sum_{i=1}^{N_E} \bar{f}_i \cdot (\frac{1}{N_P}\sum_{j=1}^{N_p} P_i^j) \quad (5)$$

$$= \frac{1}{N_P}\sum_{j=1}^{N_p}(N_E\sum_{i=1}^{N_E} \bar{f}_i \cdot P_i^j) \quad (6)$$



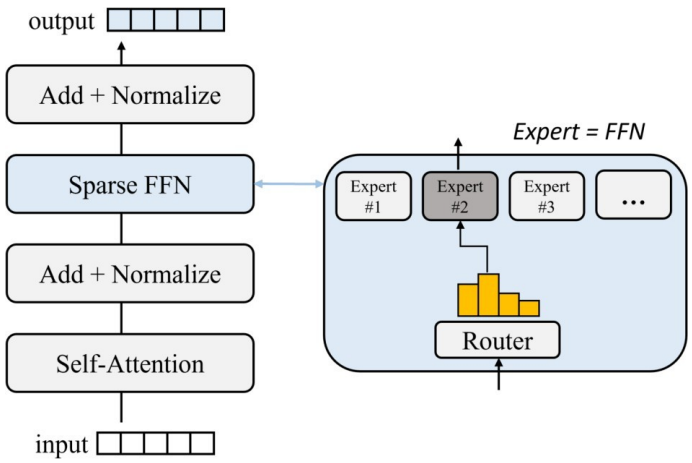
(a) Ablation of Balance Scope

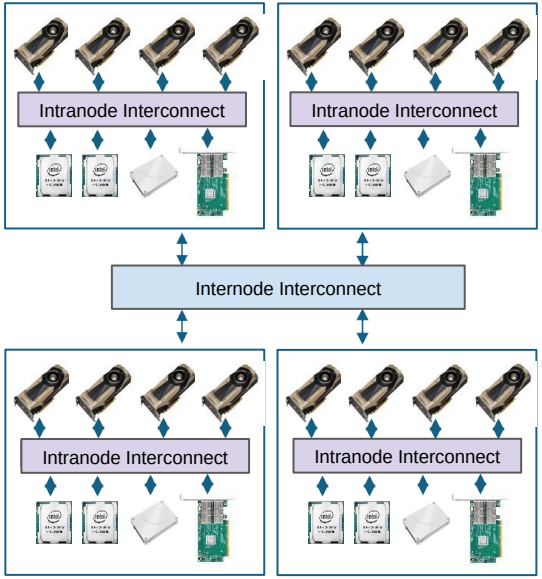


(b) Expert Frequency for Micro-Batch Balance (Left) and Global-Batch Balance (Right)

[global-batch LBL]

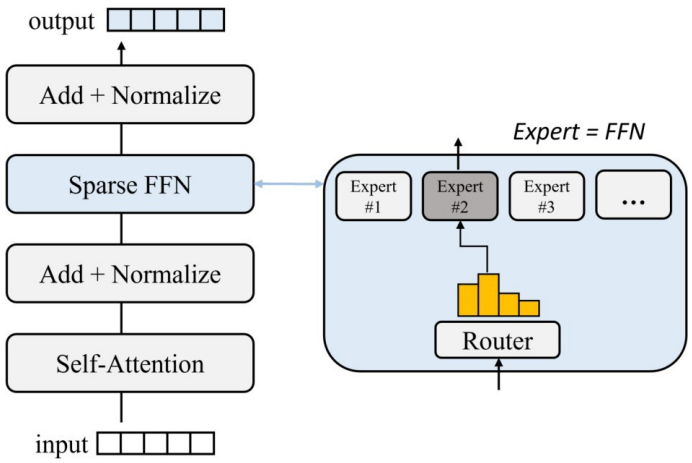
MoE Scaling Challenges on Modern Hardware with Massive Parallelism

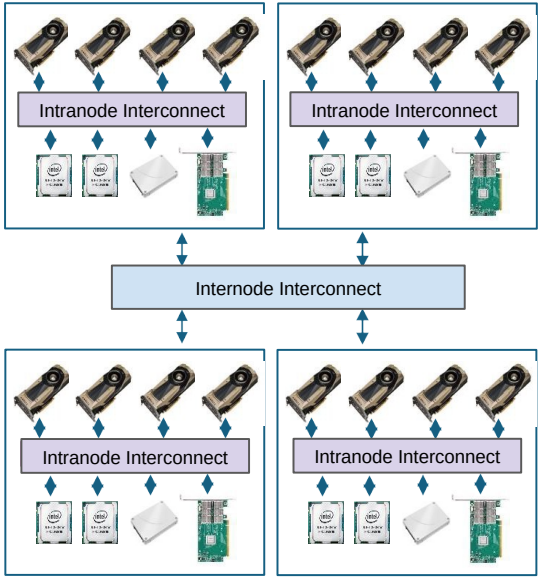




MoE Scaling Challenges on Modern Hardware with Massive Parallelism

- How to break the memory wall to enable massive MoEs?
- How to efficiently route tokens to different experts across GPUs?
- How to minimize communication overhead while achieving high per-GPU compute throughput?

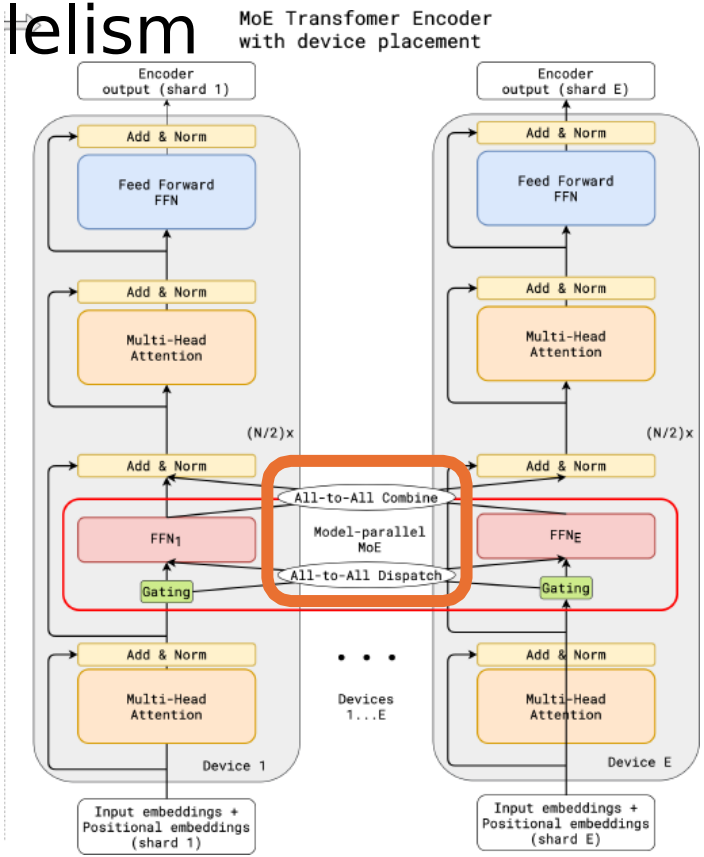




DeepSeek-MoE Expert Parallelism

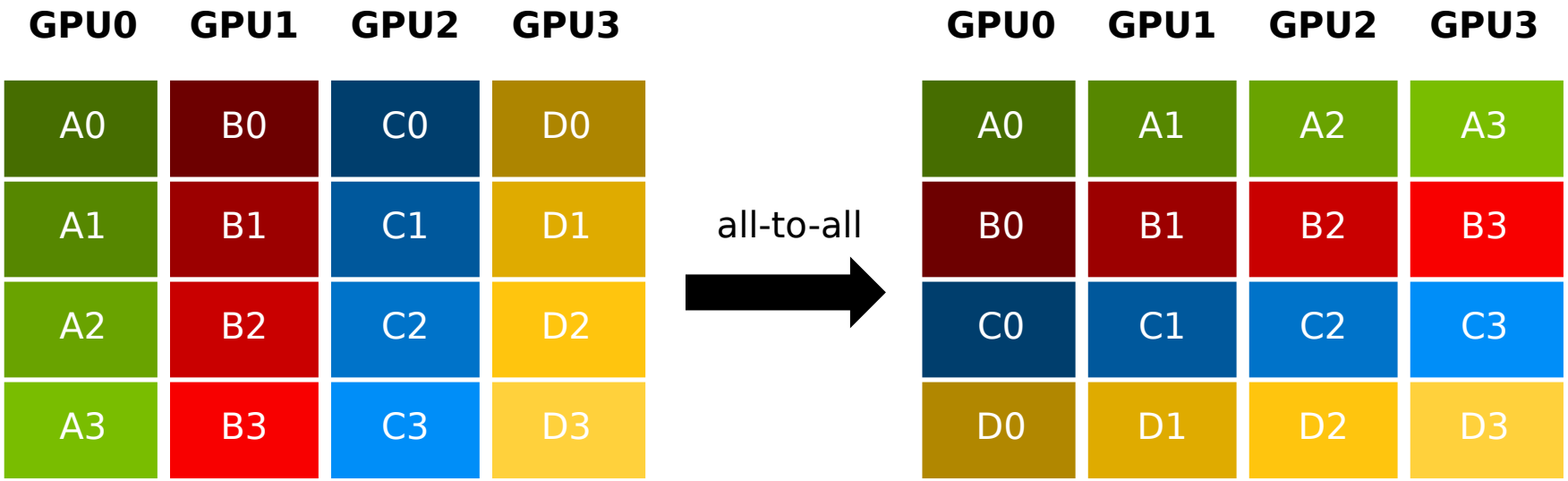
- Expert parameters – partitioned (sharded)
  - Like model parallelism (MP)
    - We'll discuss in a few lectures
  - Each expert processes a subset of tokens

- Two All-to-All(s) in forward and backward (if training)



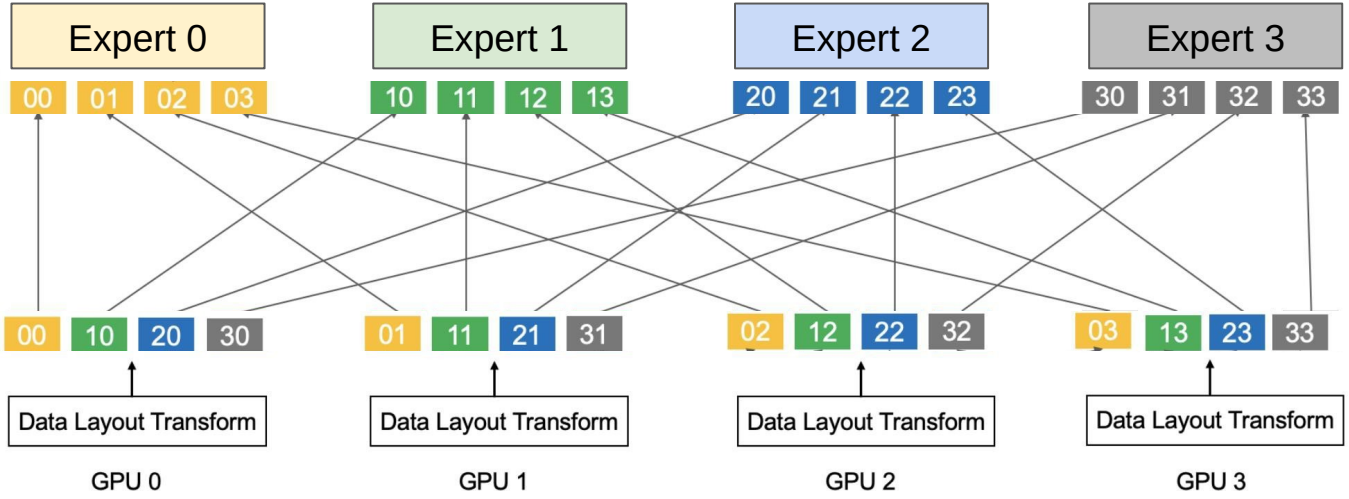
All-to-All Collective Communication

Scatter/Gather distinct messages from each participant to every other



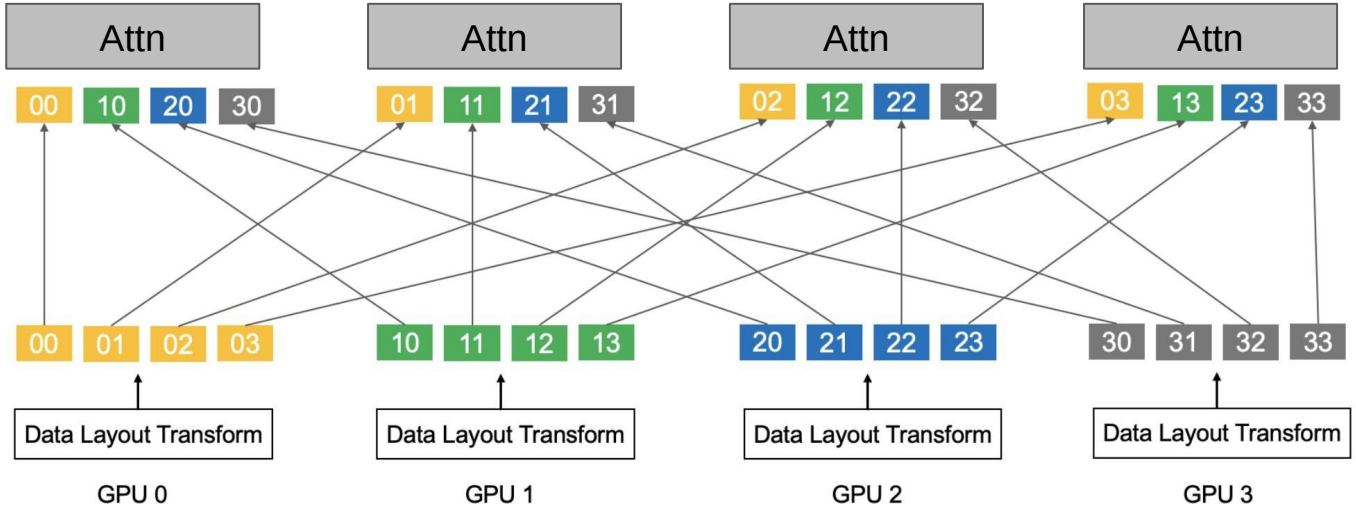
DeepSpeed-MoE Expert Parallelism

1. Gating function: decide target experts for each token
2. **Dispatch phase:**
   a. 1st layout transformation: tokens to the same target experts are grouped in a continuous memory buffer
   b. 1st All2All: dispatch tokens to their corresponding experts



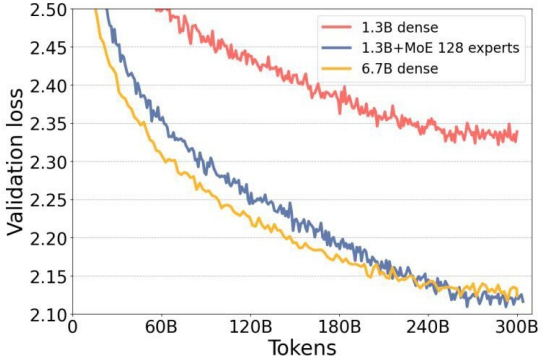
DeepSpeed-MoE Expert Parallelism

3. Expert compute: each expert process its tokens
4. **Combine phase:**
   a. 2nd All2All: combine processed tokens back to their GPUs
   b. 2nd layout transformation: restore tokens to their original positions



DeepSpeed-MoE: Cheaper GPT Model Training with MoE

- 1.3B+MoE with 128 experts, compared to 1.3B and 6.7B dense (GPT-3 like)
- 8x more parameters to same accuracy using MoE
- 5x lower training cost to same accuracy using MoE



Case	Model size	LAMBADA: completion prediction	PIQA: commonsense reasoning	BoolQ: reading comprehension	RACE-h: reading comprehension	TriviaQA: question answering	WebQs: question answering
Dense GPT:							
(1) 350M	350M	52.03	69.31	53.64	31.77	3.21	1.57
(2) 1.3B	1.3B	63.65	73.39	63.39	35.60	10.05	3.25
(3) 6.7B	6.7B	**71.94**	**76.71**	**67.03**	37.42	23.47	5.12
Standard MoE GPT:							
(4) 350M+MoE-128	13B	62.70	74.59	60.46	35.60	16.58	5.17
(5) 1.3B+MoE-128	52B	69.84	**76.71**	64.92	**38.09**	**31.29**	**7.19**

	Training samples per sec	Throughput gain/ Cost Reduction
6.7B dense	70	1x
1.3B+MoE-128	372	**5x**

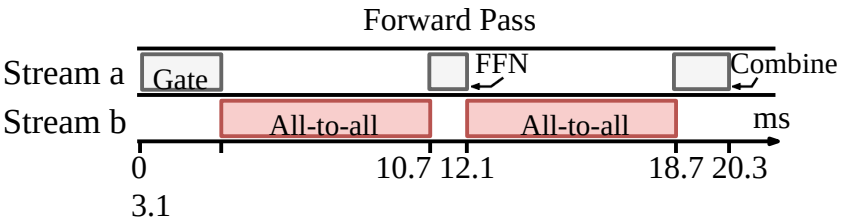
Distributed MoE is not efficient

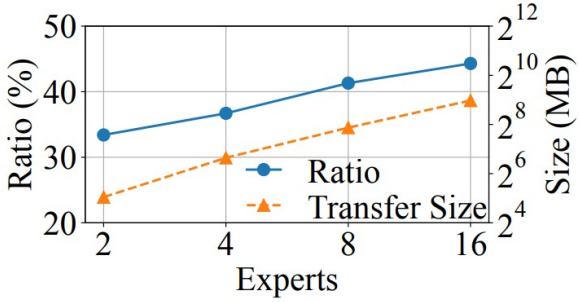
All-to-all takes an average of 34.1% of the step time.

# Experts	Model	Training (ms)		Inference (ms)	
GPUs	#Layers & Params	All-to-all	Ratio	All-to-all	Ratio
4	12L + 117M	259	36.7%	73	27.4%
	24L + 233M	589	35.4%	103	26.2%
	36L + 349M	979	38.2%	153	28.3%
16	12L + 419M	333	39.5%	102	32.5%
	24L + 838M	715	37.6%	177	31.7%
	36L + 1.2B	1145	36.8%	243	27.4%

All-to-all is the bottleneck

- Synchronous and blocking operation & large amounts of data transfer.

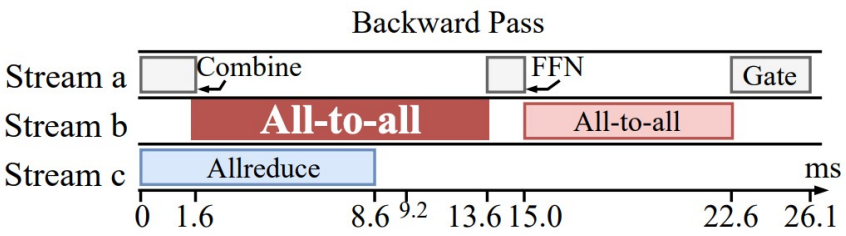




MoE Training in Data & Expert Parallelism

In backward pass,

- Allreduce: asynchronously aggregate *non-expert* gradients in a data parallel manner.

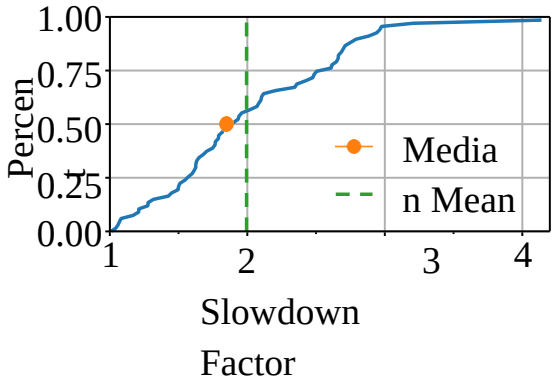


to compute *expert* gradients.

All-to-all is prolonged when it overlaps with allreduce and directly impacts step time.

Slowdown of all-to-all varies:

Median: 2x; Maximum: ~4x

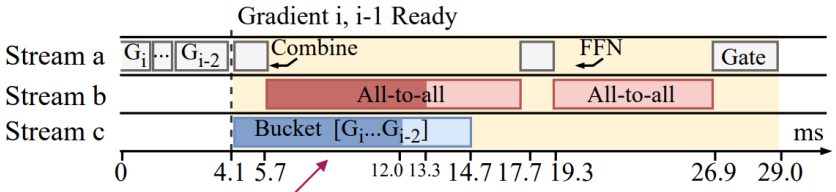


Lina: Training Optimization

Intuition: always prioritize all-to-all and avoid bandwidth sharing

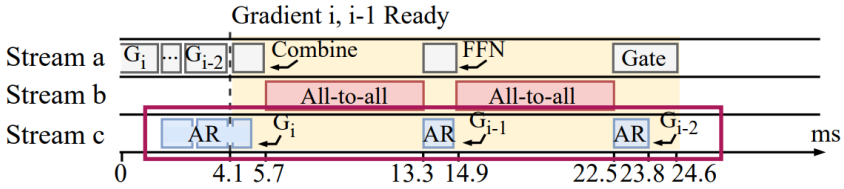
Minimize the blocking period incurred by all-to-all

Backward pass



All-reduce operation

Baseline



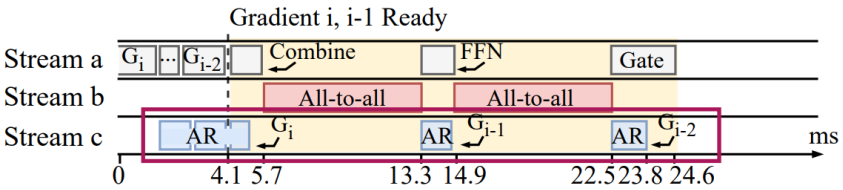
Prioritize all-to-all

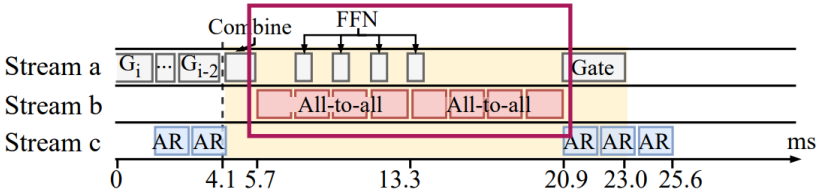
- Tensor Partitioning
  - Partition allreduce into micro-ops
  - Prioritize all-to-all whenever possible

Lina: Training Optimization

Intuition: always prioritize all-to-all and avoid bandwidth sharing.

Minimize the blocking period incurred by all-to-all



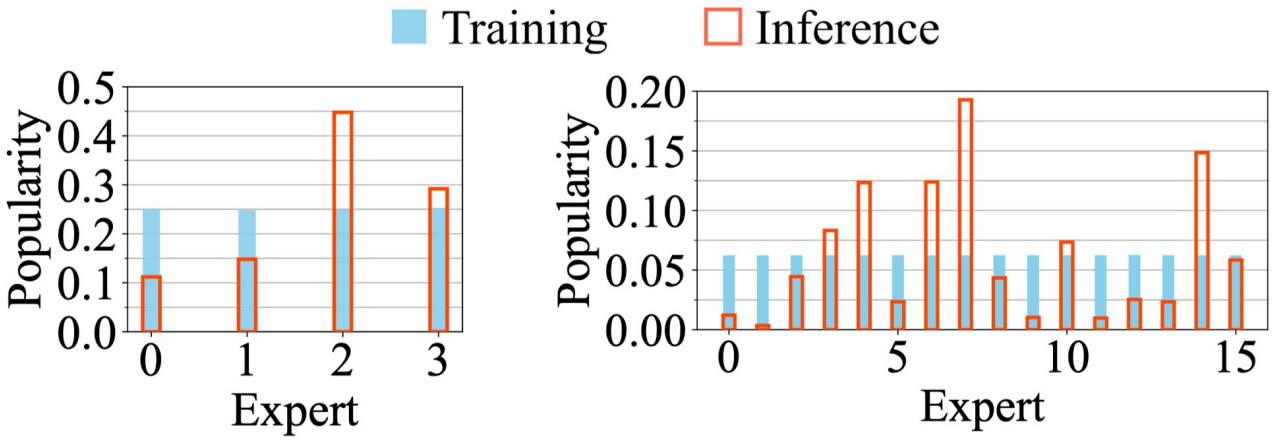


Prioritize all-to-all

- Partition all-to-all into micro-ops
- Pipelining computation and all-to-all

Lina: Inference Optimization

- During MoE training, a load balancing loss is introduced to make sure each expert gets enough training samples
- However, during MoE inference, the popularity of each expert is different



(a) 4-expert MoE.

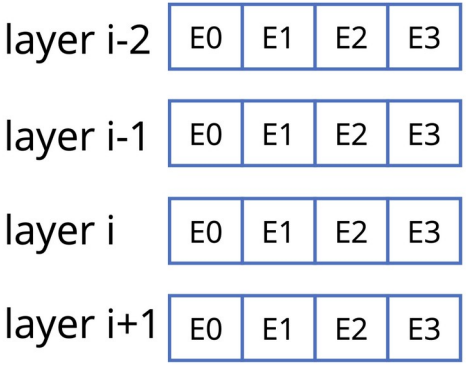
(b) 16-expert MoE.

Popular experts => More resources

How to make accurate prediction before gating happens?

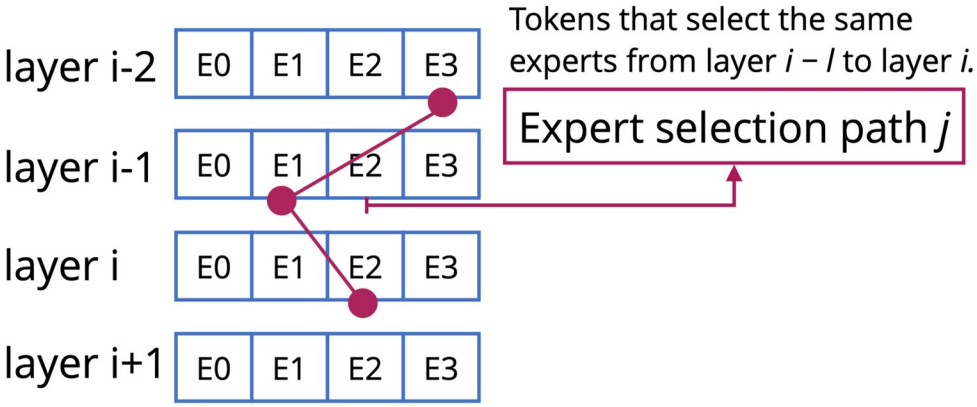
Lina: Inference Optimization

Idea: collect the expert selection distribution during training (after the load balancing loss is minimized), use this as profile for inference prediction

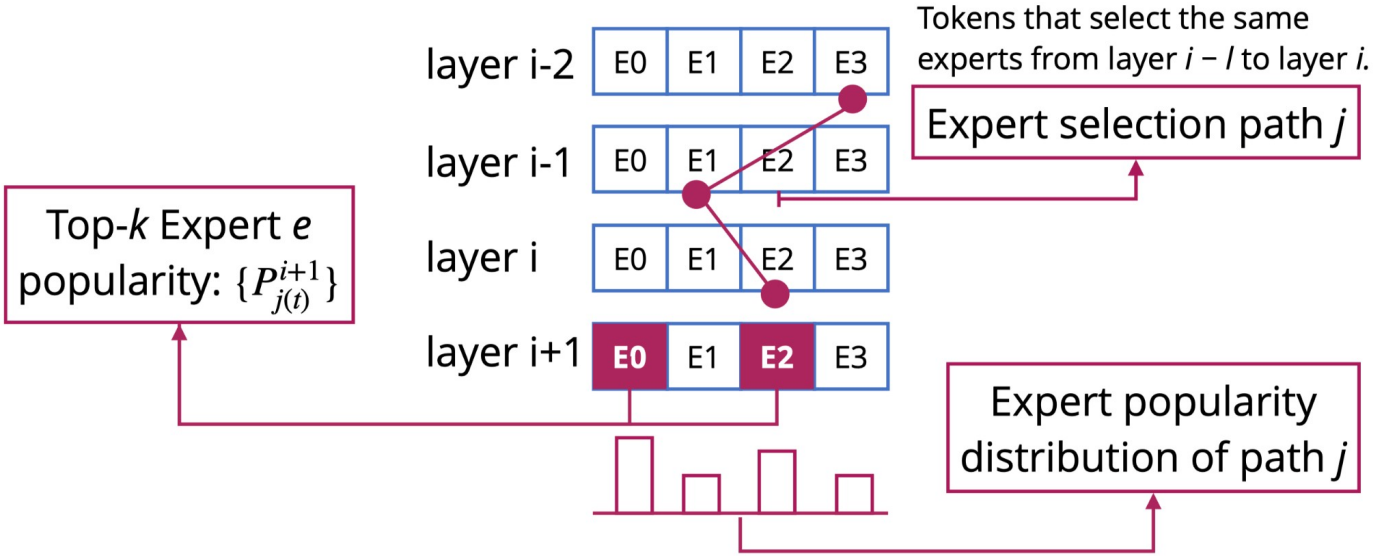


Lina: Inference Optimization

Idea: collect the expert selection distribution during training (after the load balancing loss is minimized), use this as profile for inference prediction

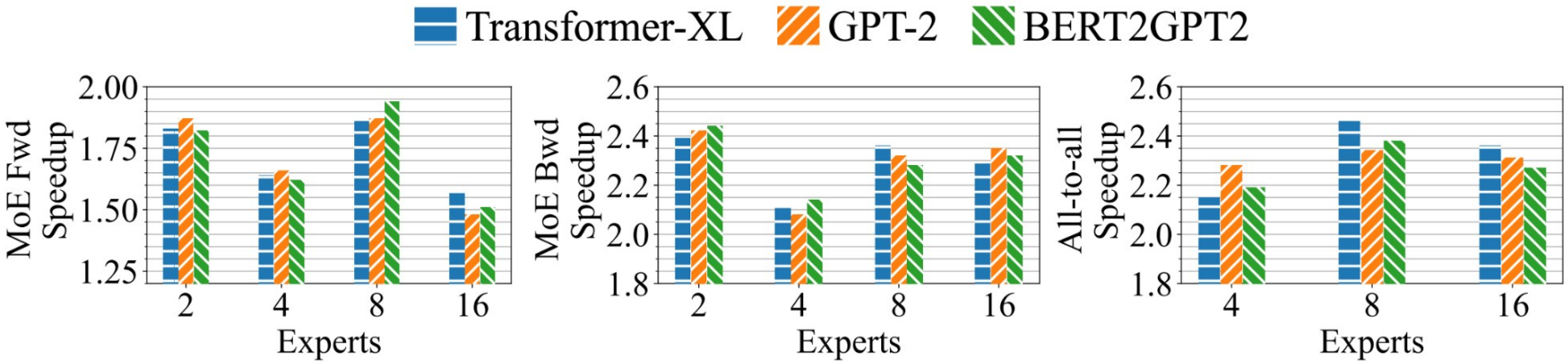


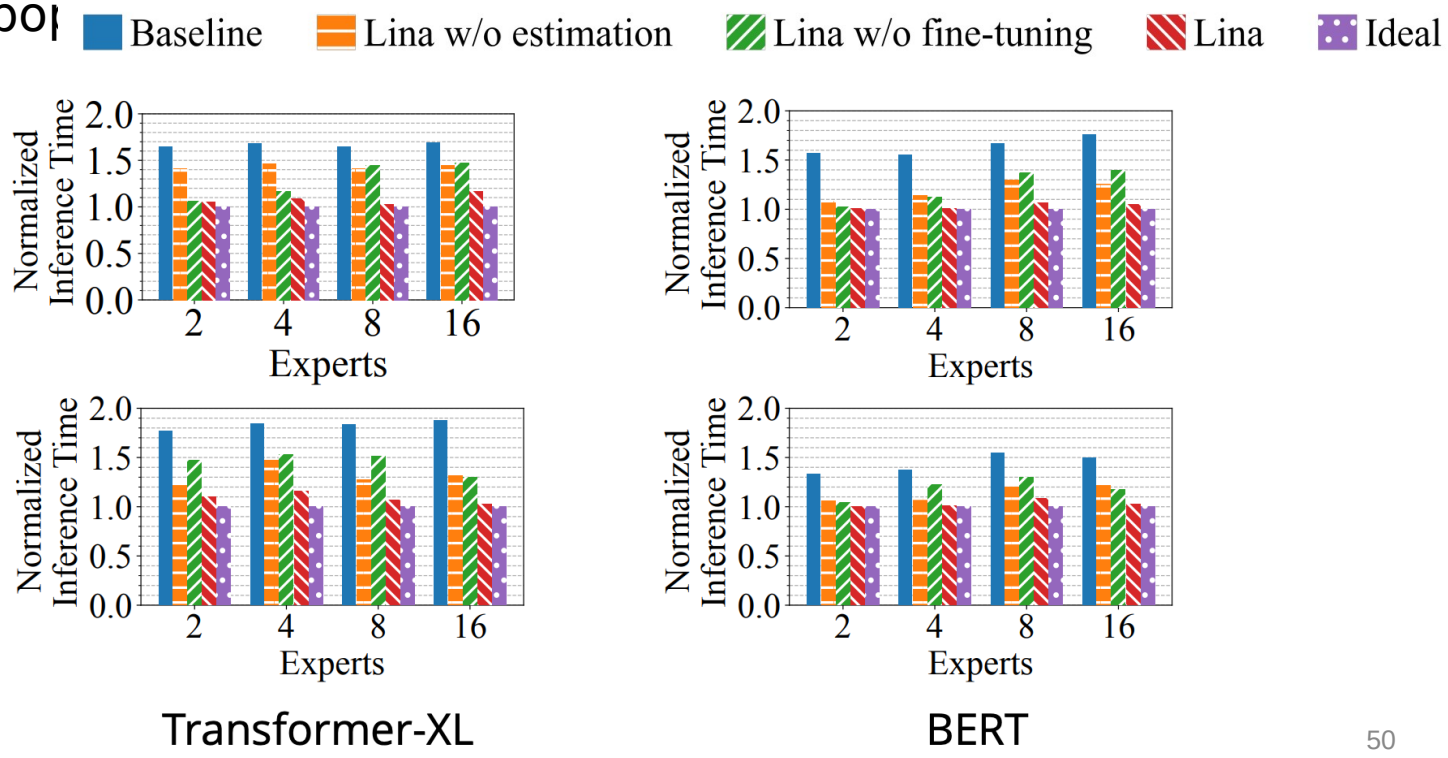
Lina: Inference Optimization



MoE Layer in Training

MoE layer forward, backward, all-to-all running time speedup over Baseline



Inference Latency

Ideal: schedule resources assuming we have prior knowledge of exact expert po